```
// ================= FULL MERN TRAVEL PLANNER (ADVANCED) =================

Features:

 JWT Auth (Login/Signup)

 CRUD Trips (Add, View, Delete, Update)

 Budget Chart (Chart.js)

 Google Maps Integration

 Tailwind UI


// ================= BACKEND =================

// Install:

// npm init -y

// npm install express mongoose cors jsonwebtoken bcryptjs dotenv


// server.js

require('dotenv').config();

const express = require('express');

const mongoose = require('mongoose');

const cors = require('cors');

const jwt = require('jsonwebtoken');

const bcrypt = require('bcryptjs');


const app = express();

app.use(cors());

app.use(express.json());


mongoose.connect('mongodb://127.0.0.1:27017/travel_planner');
```

```
// ================= MODELS =================

const UserSchema = new mongoose.Schema({
  email: String,
  password: String
});

const TripSchema = new mongoose.Schema({
  title: String,
  destination: String,
  budget: Number,
  date: String,
  userId: String
});

const User = mongoose.model('User', UserSchema);
const Trip = mongoose.model('Trip', TripSchema);

// ================= MIDDLEWARE =================

const auth = (req, res, next) => {
  const token = req.headers['authorization'];
  if (!token) return res.send('Access Denied');

  try {
    const verified = jwt.verify(token, 'secret');
    req.user = verified;
```

```
    next();
  } catch {
    res.send('Invalid Token');
  }
};

// ================= AUTH ROUTES =================

app.post('/signup', async (req, res) => {
  const hashed = await bcrypt.hash(req.body.password, 10);
  const user = new User({ email: req.body.email, password: hashed });
  await user.save();
  res.send('User Created');
});

app.post('/login', async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user) return res.send('User not found');

  const valid = await bcrypt.compare(req.body.password, user.password);
  if (!valid) return res.send('Wrong password');

  const token = jwt.sign({ id: user._id }, 'secret');
  res.send(token);
});

// ================= TRIP ROUTES =================
```

```
app.post('/add', auth, async (req, res) => {
  const trip = new Trip({ ...req.body, userId: req.user.id });
  await trip.save();
  res.send(trip);
});

app.get('/trips', auth, async (req, res) => {
  const trips = await Trip.find({ userId: req.user.id });
  res.send(trips);
});

app.put('/update/:id', auth, async (req, res) => {
  await Trip.findByIdAndUpdate(req.params.id, req.body);
  res.send('Updated');
});

app.delete('/delete/:id', auth, async (req, res) => {
  await Trip.findByIdAndDelete(req.params.id);
  res.send('Deleted');
});

app.listen(5000, () => console.log('Server running'));

// ================= FRONTEND =================
// npx create-react-app client
// cd client
```

```
// npm install axios react-chartjs-2 chart.js
// npm install -D tailwindcss

// ================= App.js =================

import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { Bar } from 'react-chartjs-2';

function App() {
  const [token, setToken] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [trips, setTrips] = useState([]);
  const [title, setTitle] = useState('');
  const [destination, setDestination] = useState('');
  const [budget, setBudget] = useState('');
  const [date, setDate] = useState('');

  const headers = { headers: { Authorization: token } };

  const signup = async () => {
    await axios.post('http://localhost:5000/signup', { email, password });
    alert('Signup success');
  };

  const login = async () => {
```

```
  const res = await axios.post('http://localhost:5000/login', { email, password });
  setToken(res.data);
};

const fetchTrips = async () => {
  const res = await axios.get('http://localhost:5000/trips', headers);
  setTrips(res.data);
};

const addTrip = async () => {
  await axios.post('http://localhost:5000/add', { title, destination, budget, date }, headers);
  fetchTrips();
};

const updateTrip = async (id) => {
  const newTitle = prompt('Enter new title');
  await axios.put(`http://localhost:5000/update/${id}`, { title: newTitle }, headers);
  fetchTrips();
};

const deleteTrip = async (id) => {
  await axios.delete(`http://localhost:5000/delete/${id}`, headers);
  fetchTrips();
};

useEffect(() => {
  if (token) fetchTrips();
```

```
  }, [token]);

  const chartData = {
    labels: trips.map(t => t.title),
    datasets: [{ label: 'Budget', data: trips.map(t => t.budget) }]
  };

  return (
    <div className="p-5">
      <h1 className="text-2xl font-bold">Travel Planner</h1>

      {!token && (
        <>
          <input placeholder="Email" onChange={e => setEmail(e.target.value)} />
          <input placeholder="Password" type="password" onChange={e =>
setPassword(e.target.value)} />
          <button onClick={signup}>Signup</button>
          <button onClick={login}>Login</button>
        </>
      )}

      {token && (
        <>
          <input placeholder="Title" onChange={e => setTitle(e.target.value)} />
          <input placeholder="Destination" onChange={e => setDestination(e.target.value)} />
          <input placeholder="Budget" onChange={e => setBudget(e.target.value)} />
          <input placeholder="Date" onChange={e => setDate(e.target.value)} />
          <button onClick={addTrip}>Add</button>
```

```
        <Bar data={chartData} />


        {trips.map(t => (
          <div key={t._id} className="border p-2 m-2">
            <h3>{t.title}</h3>
            <p>{t.destination}</p>
            <p>₹{t.budget}</p>
            <button onClick={() => updateTrip(t._id)}>Edit</button>
            <button onClick={() => deleteTrip(t._id)}>Delete</button>
          </div>
        ))}
      </>
    )}
  </div>
 );
}

export default App;
```